

Skynet MMO 学习课程第三课：完整 MMO 业务闭环与一致性

基线：Skynet v1.8.0；Bundled Modified Lua 5.4.7；Sproto + 2-byte big-endian framing。

前置能力：能够解释 Service/Mailbox/Worker/coroutine，能对 `skynet.call` 做逐 yield 审查。

本课目标：把第二课的运行语义应用到登录、重连、进场、移动、AOI、战斗和持久化，形成一套
可用于真实 MMO 代码评审的一致性方法。

SKYNET MMO LEARNING

目录

0. 本课不是业务功能导览，而是状态机审计	5
1. 先建立全局状态所有权表	6
2. 一次完整会话的宏观时序	7
3. 协议边界：TCP 字节流不等于业务消息	8
4. 启动就绪：为什么 Gate 必须最后打开	9
5. TCP 建连：Gate 与 Watchdog 的职责分界	10
5.1 为什么同时需要 fd 和 connection_id	10
6. 登录状态机：先发布状态，再进入第一个 yield	11
6.1 每次恢复都要做 alive 校验	11
7. Authentication：教学桩与生产信任边界	12
8. 重复登录：PlayerMgr 如何保证一个玩家一个 Agent	13
8.1 分片锁的成本模型	13
9. Agent 创建与加载：哪些事实已经提交	14
9.1 当前加载链的同步深度	14
10. bind_client 与顶号：玩家身份和连接身份分离	15
11. 进入世界：SceneMgr 管生命周期，Scene 管实时状态	16
11.1 Scene.enter 的原子提交段	16
12. 登录 Response 与 AOI Push：客户端可见顺序也是一致性	17
12.1 Response 排队不等于客户端已经收到	17
13. Gate.forward：从控制面切换到玩家数据面	18
14. 玩家请求串行：业务事务边界不等于 Service callback	19
14.1 什么可以并行，什么应保持串行	19
15. 权威移动：一次跨 Service 的两阶段事实	20
15.1 Token Bucket 为什么在 Scene	20
15.2 客户端预测的正确角色	20
16. AOI：可见集合是一份需要维护的不变量	21
16.1 AOI 更新顺序	21
17. 战斗、死亡和重生：先提交状态，再广播结果	22
17.1 客户端观察顺序与业务事实	22

17.2 Respawn Timer 的代号问题	22
18. 主动 logout：为什么关闭要延迟一个 tick	23
19. 被动断线与 60 秒重连窗口	24
19.1 Grace Window 的产品语义	24
20. Offline Pipeline：退出并不是一个原子操作	25
20.1 当前实现的一个重要竞态窗口	25
21. Storage：同一玩家路由有序，不代表已经持久可靠	26
21.1 Versioned Snapshot 的最小模型	26
22. 一致性不是一个词：为每条数据声明模型	27
23. 超时、重试和幂等：三者必须一起设计	28
24. 场景切换：当前未实现，但应该怎样推导协议	29
25. 故障矩阵：先定位事实，再决定恢复	30
26. 实验一：逐断点走完整登录链	31
26.1 Watchdog 轮	31
26.2 PlayerAgent 轮	31
26.3 Scene 轮	31
27. 实验二：用两个客户端验证顶号与 Generation	32
28. 实验三：重连窗口与当前缺口	33
29. 实验四：验证客户端可见顺序	34
30. 测试金字塔：每层证明不同性质	35
31. 业务链代码评审模板	36
31.1 消息和上下文	36
31.2 状态和提交	36
31.3 Coroutine	36
31.4 失败和恢复	36
31.5 可观测和验证	36
32. 当前工程已经保证与尚未保证的边界	37
已经清晰表达	37
仍是生产差距	37
33. 自测题 (1-5)	38
1. 为什么 login Response 没收到，不能证明登录没有执行？	38

2. PlayerAgent 与 Scene 都保存坐标，为什么当前仍不是双主？	38
3. AUTHING 为什么必须在 Auth call 前写入？	38
4. Gate.forward 后 Watchdog 是否退出了连接生命周期？	38
5. serial 为什么不能自动保护 bind_client 与 offline 的交错？	38
33.1 自测题 (6-10)	39
6. Scene.enter 返回 snapshot 时，玩家在 Scene 中是否已经存在？	39
7. 为什么同 player_id 路由到同一 Storage Worker 仍不足以防止旧快照覆盖？	39
8. Monster respawn timer 何时需要 Generation？	39
9. 为什么关键经济操作不能照搬 move 的“不重试”策略？	39
10. 如何证明顶号后的旧 close 没有踢掉新连接？	39
34. 本课源码索引	40
35. 本课完成标准与第四课入口	41

0. 本课不是业务功能导览，而是状态机审计

只看正常路径，这个工程很简单：客户端登录、移动、攻击、退出。但真实 MMO 故障通常发生在路径之间：认证等待时断线、顶号与旧 close 交错、进场成功但响应未送达、移动已提交而持久快照未更新、Scene leave 后 DB 保存失败。

因此本课固定用五个维度阅读每条业务链：

Message 消息从哪个 Service 到哪个 Service
Ownership 哪个 Service 对哪份状态拥有最终写权
Yield coroutine 在哪里挂起，恢复时什么可能已变化
Commit 哪一步之后业务事实已经不可假装没有发生
Observation 客户端、日志、Trace、状态快照和测试分别能看到什么

本课不会把样例当前能运行的行为包装成“已经达到生产级”。每一节都区分三类结论：

- 当前代码已经明确保证的性质。
- 当前 Demo 为教学而采用的简化。
- 真实项目必须补齐的协议、版本或补偿机制。

1. 先建立全局状态所有权表

一致性的第一步不是选数据库或加锁，而是回答“谁能最终决定”。

状态	Owner	其他副本的性质	当前提交点
fd 与登录阶段	Watchdog	Gate 保存转发信息	<code>connections[fd]</code> 状态变化
fd -> Agent 数据面路由	Gate	Watchdog 只控制生命周期	<code>gate.forward</code> 更新 connection
player_id -> PlayerAgent	PlayerMgr	Watchdog 只缓存本次结果	<code>players[player_id] = agent</code>
持久角色快照	PlayerAgent	Storage 保存副本	PlayerAgent 业务状态更新
权威实时坐标和移动额度	Scene	Agent 的坐标是持久快照	Scene 无 yield 的 move commit
AOI Grid 与可见集合	Scene	客户端只持表现副本	<code>grid/entities/visible</code> 同步更新
怪物 HP、死亡和重生	Scene	Agent 只转发 Push	Scene attack/respawn callback
最终落盘数据	Storage Worker / DB	Agent 持运行时新版本	DB UPDATE 成功

这里最重要的不是“数据存了几份”，而是每份副本有没有角色定义。

Authority 可以校验并决定事实
 Snapshot 只在 Authority 成功后更新，用于恢复或持久化
 Cache 可以丢弃并从 Owner 重建
 View 面向客户端的派生表现，不参与服务端判定

看到同名字段时不能直接判定双主。例如 Scene 与 PlayerAgent 都有 `x/y`，但只要写入协议始终是“Scene 先决定，Agent 后复制”，仍然可以保持单一权威。真正危险的是双方都能独立接受写入。

2. 一次完整会话的宏观时序

先把一名玩家从 TCP connect 到最终 offline 放在一张图里：

```

Client
-> Gate connect
-> Watchdog: CONNECTED, connection_id++
-> Gate accept/openclient

Client login
-> Watchdog: AUTHING
-> Auth.verify [yield]
-> PlayerMgr.login [yield]
  -> new PlayerAgent [yield]
    -> Agent.load -> StorageMgr [yield]
-> Agent.bind_client [yield]
-> Agent.enter_world -> SceneMgr [yield]
  -> Scene.enter [yield]
-> Gate.forward [yield]
-> login Response queued
-> Agent.client_ready -> AOI snapshot Push

Client move/attack
-> Gate redirect PTYPE_CLIENT directly to Agent
-> Agent serial request queue
-> Scene move/attack [yield at Agent]
-> Scene authoritative commit [no yield inside commit]
-> Agent/client Response + AOI Push

TCP close
-> Gate -> Watchdog
-> Agent.client_closed
-> 60 s reconnect grace timer
-> Scene.leave [yield]
-> Storage.save [yield]
-> PlayerMgr.remove [yield]
-> Agent exit
  
```

图中“同一行”不代表同一事务。“Scene 已提交移动”与“客户端收到成功响应”之间仍有多个可能失败的环节。设计补偿前必须先确定每个 commit point。

3. 协议边界：TCP 字节流不等于业务消息

本工程在 TCP 之上使用 2-byte big-endian 长度前缀：

```

+-----+-----+
| payload length: 2 B | Sproto payload: N B |
+-----+-----+
    >s2          max 65535 B

```

lua-lib/protocol/frame.lua 的 `string.pack(">s2", payload)` 负责发送端 framing；官方 Gate 的 `header = 2` 负责收包。TCP 可能拆包或粘包，长度帧才定义应用消息边界。

Sproto `.package` 又在 payload 内提供：

```

type    协议编号
session 客户端 RPC 关联号
body    request/response fields

```

不要把三种 session 混为一谈：

名称	所在层	用途
Client Sproto session	Client <-> Server protocol	匹配业务 Request/Response
Skynet message session	Service <-> Service	匹配 <code>skynet.call/retpack</code>
Gate forward 的 session 参数	<code>PTYPE_CLIENT</code> 投递	本项目用它携带 fd，不表示 RPC

PlayerAgent 的 client protocol dispatcher 第一形参写作 `fd`，实际位置是 Skynet dispatch 的 session。它必须调用 `skynet.ignoreret()`，因为 Gate redirect 不是一个等待 `skynet.ret()` 的同步 RPC。

4. 启动就绪：为什么 Gate 必须最后打开

`service/main.lua` 按依赖顺序初始化：Protocol、Config、Storage、Scene、PlayerMgr、Auth、Watchdog/Gate。公开监听端口最后打开。

这相当于明确 Ready Barrier：

```
dependencies created
-> storage workers connected
-> scene initialized
-> managers receive dependency addresses
-> gate listen
-> startup complete
```

若先 listen，再异步初始化 Storage 或 Scene，客户端可能在服务“进程活着但业务未就绪”的窗口进入。真实部署中还要区分：

- Liveness：进程/Runtime 是否活着。
- Readiness：是否具备接受新登录的全部依赖。
- Draining：是否停止接新连接但继续处理已有玩家。

当前 `main.lua` 的同步启动链能把配置错误和连接失败暴露在 Ready 之前，但还没有独立健康检查接口、优雅摘流和启动超时，这些属于第四课的生产工程范围。

5. TCP 建连：Gate 与 Watchdog 的职责分界

官方 `service/gate.lua` 拥有 Socket 收包与 fd 转发表；自定义 Watchdog 拥有登录前连接状态机。

连接事件路径：

```
Socket Thread
-> gateserver
-> gate.handler.connect(fd, addr)
-> Gate.connection[fd] = {fd, ip}
-> send Watchdog "socket/open"
-> Watchdog.connections[fd] = {
    id = new_connection_id(),
    state = "CONNECTED",
    agent = nil
}
-> call Gate.accept(fd)
-> gateserver.openclient(fd)
```

Gate 先接管 fd，但显式 `accept/openclient` 后才开始业务读取。这使 Watchdog 能先建立状态再放行数据。

5.1 为什么同时需要 fd 和 connection_id

fd 只是 OS 可复用的槽位。逻辑连接身份应为：

```
ConnectionIdentity = (fd, connection_id)
```

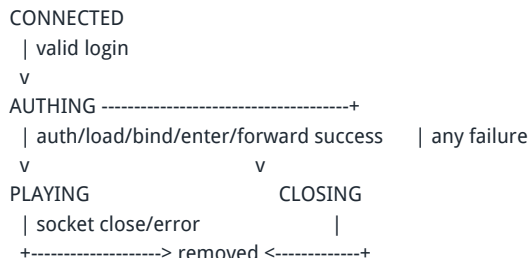
延迟 kick、close callback 和 Agent 通知如果只携带 fd，旧连接的消息可能作用于已经复用该 fd 的新连接。`connection_id` 就是 Generation。

6. 登录状态机：先发布状态，再进入第一个 yield

Watchdog 只允许 **CONNECTED** 状态接收第一包，并要求它是合法 **login**。在调用 Auth 之前先执行：

```
c.state = "AUTHING"
local auth_ok = skynet.call(auth_service, "lua", "verify", ...)
```

准确状态图：



如果把 **AUTHING** 放在 **call** 之后，第一条 coroutine 挂起时，第二个数据包仍会看到 **CONNECTED** 并重复发起登录。这是典型的“Service 内 coroutine 重入”，不是多线程 Data Race。

6.1 每次恢复都要做 alive 校验

Watchdog 在 Auth、PlayerMgr、bind、enter 等 call 后反复执行：

```
if not alive(fd, c) then
    return
end
```

alive 比较 **connections[fd] == c**，同时验证 fd 映射仍指向原 connection object。检查不能只放在链尾，因为每个 await 都是一个新的失效窗口。

7. Authentication：教学桩与生产信任边界

当前 Auth 仅验证：

```
token == "dev:" .. tostring(player_id)
```

它只用于打通业务链，不能作为生产认证。生产 GameServer 不应相信客户端自报 player_id；常见协议是 Login/Center 签发短时、一次性或可撤销的 ticket，GameServer 验证并消费。

认证设计至少要回答：

- Token 绑定哪个 account/player/server/region？
- 是否有 expiry、nonce 和 replay protection？
- 验证依赖失败时是拒绝登录还是降级？
- 顶号权限和封禁状态在哪里决定？
- 敏感字段是否进入普通日志？

不要因为 Auth 是一个 Service 就自动获得安全性。Service 只提供隔离和消息边界，信任模型仍由协议、密钥和状态所有权决定。

8. 重复登录：PlayerMgr 如何保证一个玩家一个 Agent

PlayerMgr 拥有 `player_id -> PlayerAgent Address` 映射。`CMD.login` 使用 64 个 `skynet.queue` 分片锁：

```
lock = login_locks[player_id % 64 + 1]
lock {
    recheck players[player_id]
    newservice PlayerAgent      [yield]
    Agent.load                  [yield]
    players[player_id] = agent
}
```

双重检查发生在锁内，而不是只在锁外检查一次。原因是 `newservice/load` 都会 `yield`，两条同玩家登录 `coroutine` 否则可能各自创建 Agent。

8.1 分片锁的成本模型

- 同一 `player_id` 必须串行，这是正确性要求。
- 不同 `player_id` 若哈希到同一槽，也会在创建阶段短暂串行，这是固定 64 锁换取有界内存的代价。
- 登录完成后的 `move/attack` 不经过 PlayerMgr，因此 Manager 不在高频数据面。

真实项目可按账号、角色或 Login Transaction ID 建模，但不应把“加一个全局登录锁”当作最简单方案；它会把所有慢 Storage Load 放进同一临界区。

9. Agent 创建与加载：哪些事实已经提交

新登录路径中：

```
PlayerMgr newservice Agent
-> Agent Lua State created
-> call Agent.load
    -> StorageMgr
    -> selected Worker
    -> load snapshot
-> players[player_id] = agent
```

在 `players[...] = agent` 之前，Agent 只是候选实例；load 失败会收到 `shutdown`。映射写入之后，PlayerMgr 才对外声明这个 Agent 是该玩家的在线 owner。

Memory Worker 在 load 时 clone table。这不是为了跨线程锁，而是为了避免调用方意外持有 Storage 内部可变 table。在不同 Lua State 间，Skynet 序列化本来也会复制消息；保留 clone 让 Memory 实现的契约更明确，也便于脱离 Skynet 单测时保持边界。

9.1 当前加载链的同步深度

```
Watchdog -> PlayerMgr -> PlayerAgent -> StorageMgr -> StorageWorker
```

链上每层都可能排队。登录 Tail Latency 必须能拆成 Auth、Agent Create、Storage Queue、DB Query、Scene Enter 等区间；只记录总耗时无法判断扩容哪一层。

10. bind_client 与顶号：玩家身份和连接身份分离

同一 PlayerAgent 可被新连接复用。bind_client 若发现旧 fd，会先异步通知旧 Watchdog kick，再写入新的：

```
client_fd
client_watchdog
client_connection_id
client_ready = false
pending_pushes = {}
offline_version++
```

关键语义：

- PlayerAgent 身份跟随 player_id，不跟随某次 TCP 连接。
- Connection Identity 是 (watchdog, fd, connection_id)。
- 旧连接的延迟 client_closed 到达时，Agent 会同时比较 fd 和 connection_id，不能清除新绑定。
- offline_version++ 使旧的 Grace Timer 失效。

生产实现通常还会向旧客户端发送明确 kick reason，并定义新连接何时可见为成功。当前示例采用“新连接优先，旧连接异步关闭”的策略。

11. 进入世界：SceneMgr 管生命周期，Scene 管实时状态

PlayerAgent 使用持久快照构造 enter info，再 call SceneMgr：

```
Agent.enter_world
-> SceneMgr.enter(info)
  -> get_scene(scene_id)
  -> Scene.enter(info)
<- ok, scene_address, snapshot
-> Agent.scene_service = scene_address
-> snapshot append to pending_pushes
```

SceneMgr 的 `create_lock` 防止两个 coroutine 在 `newservice/init` yield 期间创建同一 `scene_id`。它负责生命周期和 Address Discovery；Scene 创建后，PlayerAgent 直接持有地址，高频 move/attack 不再经过 Manager。

11.1 Scene.enter 的原子提交段

Scene 在无 yield 的 handler 中：

1. 校验该 player 不存在。
2. 从持久坐标创建移动额度状态。
3. 写入 `entities` 与 Grid。
4. 计算 `visible[player_id]`。
5. 更新其他玩家的对称可见关系并发送 enter Push。
6. 返回新玩家自己的 snapshot。

其中 `skynet.send` 不会挂起当前 coroutine，所以 Scene 内状态提交保持不可插入。返回 snapshot 发生在 Scene 已经承认玩家存在之后。

12. 登录 Response 与 AOI Push：客户端可见顺序也是一致性

Scene.enter 会把附近实体 snapshot 返回 Agent。Agent 此时还没有对客户端宣布登录成功，因此先把 snapshot 放入 `pending_pushes`。

Watchdog 随后按顺序执行：

```
Gate.forward(fd -> Agent)
frame.write(login response)
send Agent.client_ready
```

Agent 收到 `client_ready` 后才 flush `pending_pushes`。目的不是服务端状态正确，而是客户端观察顺序正确：

```
login success Response
before
entity_enter snapshot Push
```

否则客户端可能在创建本地 Player/Scene Context 前先收到 AOI Entity。真实客户端通常也要用 Scene Epoch、Snapshot Sequence 或 Loading Barrier 防止网络重排和跨场景旧包；当前样例只演示最小顺序屏障。

12.1 Response 排队不等于客户端已经收到

`socket.write` 成功最多说明数据进入发送路径。紧接着断线时，服务端可能已完成登录 commit，但客户端从未观察到 Response。恢复协议必须允许客户端重新查询当前事实，而不能假设“没收到响应就一定没执行”。

13. Gate.forward：从控制面切换到玩家数据面

登录前：

```
Gate -> copy payload -> Watchdog SOCKET.data
```

登录后：

```
Gate.handler.message  
-> skynet.redirect(agent, client, "client", fd, msg, sz)  
-> PlayerAgent PTYPE_CLIENT dispatcher
```

这一步移除了 Watchdog 的高频转发开销。Watchdog 继续管理连接 close/error/kick，但 move/attack 数据直接进入 PlayerAgent Mailbox。

官方 Gate 在 redirect 模式复用消息内存，避免先转成 Lua string 再复制；未绑定 Agent 时则 `skynet.tostring` 复制给 Watchdog 并 `skynet.trash` 原消息。这里涉及消息所有权：forward 模式下谁释放 payload，必须遵循官方 gateserver contract，不能随意多 free 或漏 free。

14. 玩家请求串行：业务事务边界不等于 Service callback

每个 `PTYPE_CLIENT` 包会创建一条消息 coroutine。PlayerAgent 用同一个 `serial = skynet.queue()` 包裹所有客户端业务请求：

```
request #1: move -> call Scene -> yield
request #2: attack -> wait on serial
request #3: move -> wait on serial
```

这样同一玩家的 Request/Response 顺序清晰，`REQUEST.move` 跨 Scene RPC 仍处于玩家业务临界区。

但 `serial` 没有覆盖所有 Agent 入口：`CMD.bind_client`、`CMD.client_closed`、`CMD.push`、`timeout callback` 仍可能与客户端请求 coroutine 交错。评审时必须列出全部入口，而不是只看 `REQUEST table`。

14.1 什么可以并行，什么应保持串行

- 不同玩家的 PlayerAgent 可并行。
- 不同 Scene Instance 可并行。
- 同一玩家有顺序依赖的经济、任务、装备操作默认串行。
- 纯只读查询可以考虑快照或并行，但要证明其观察语义。
- Scene 内不同实体并不会自动并行；这是单 Actor 热点的容量边界。

15. 权威移动：一次跨 Service 的两阶段事实

移动请求先在 Agent 做类型校验，然后 call Scene：

```

Agent stage:
  validate x/y integer
  call Scene.move           [yield]

Scene commit stage, no yield:
  find entity
  validate map bounds
  refill/consume move credit
  move Grid
  set authoritative x/y
  diff visible sets
  send AOI Push
  retpack accepted x/y

Agent resume stage:
  player.x/y = accepted x/y    persistence snapshot
  encode Sproto Response
  
```

这里有两个不同 commit：Scene realtime commit 和 Agent snapshot commit。Scene 是 Authority，因此 Scene 成功而 Agent 在 Response 前退出时，实时事实已经发生；不能通过让 Agent 先写坐标来“避免不一致”，那会把拒绝路径变成双主。

15.1 Token Bucket 为什么在 Scene

移动额度依赖权威坐标和服务端时间，必须与 Grid/AOI 提交处于同一无 yield 段。放在 Agent 或独立 Movement Service 都会引入跨 Actor 的读改写协议。

15.2 客户端预测的正确角色

客户端可以预测表现，但服务端 Response 才确认位置。收到 `OUT_OF_BOUNDS/MOVE_TOO_FAST` 后应回滚或平滑纠正到最近确认坐标。预测不改变 Authority；它只改变玩家看到结果的时机。

16. AOI：可见集合是一份需要维护的不变量

Grid Query 是粗过滤，距离平方是精确规则：

```
candidates = current cell + 8 neighbors
visible = candidates where distance_sq <= view_radius^2
```

移动前后集合差：

```
old - new    entity_leave
new - old    entity_enter
old & new    entity_move to persistent observers
```

玩家之间的可见关系要求对称：若 A 的 `visible[A]` 包含 B，B 的集合也应包含 A。Monster 不需要自己的客户端视图，因此只维护玩家视角。

16.1 AOI 更新顺序

Scene 先提交 Grid 与 `player.x/y`，再计算新集合，随后发送 Push。Push 是派生输出；即使某个 Agent 已掉线，Scene 的 Authority 也不能回滚。

真实项目通常为 Push 增加 Scene Epoch 和 Entity Generation，解决以下陈旧消息：

- 玩家已切图，旧 Scene 的 `entity_move` 才到。
- Entity ID 被复用，旧 death/leave 作用于新实例。
- 客户端重连后仍收到旧连接时期排队数据。

17. 战斗、死亡和重生：先提交状态，再广播结果

`Scene.CMD.attack` 在同一无 `yield` 段完成：

```
validate attacker/target/range
-> target.hp -= damage
-> send entity_hp to observers
-> if hp == 0:
    send monster_dead
    send entity_leave
    remove from visible/Grid/entities
    schedule respawn timer
-> return attack Response
```

目标 HP 与攻击距离都属于 Scene，因此 PlayerAgent 不能先计算伤害再通知 Scene。

17.1 客户端观察顺序与业务事实

当前实现通常让观察者依次看到 HP、dead、leave，攻击者还会收到 attack Response。因为 Push 与 Response 可能来自不同 Service/coroutine，客户端不应仅靠到达顺序推导唯一事实；生产协议可使用 Combat Event Sequence、Tick 或 Entity Version。

17.2 Respawn Timer 的代号问题

当前教学版本用相同 monster id，timer 到期直接 `spawn_monster(cfg)`，且函数先检查实体是否已存在。若未来支持动态重置、跨线迁移或重复计划，必须为 Spawn Instance 增加 Generation；否则旧 timer 可能复活已经被新状态替代的实体。

18. 主动 logout：为什么关闭要延迟一个 tick

`REQUEST.logout` 先返回 `{code = 0}`，同时安排一个 tick 后通知 Watchdog kick：

```
skynet.timeout(1, function()
    if client_fd == fd and client_connection_id == connection_id then
        skynet.send(watchdog, "lua", "kick", fd, connection_id, "LOGOUT")
    end
end)
```

目的：让当前 dispatcher 有机会先编码并排队 logout Response，再关闭 Socket。timer callback 又比较 `fd + connection_id`，避免旧 logout timer 踢掉期间新绑定的连接。

即便如此，Response 仍是 best effort：TCP close、进程退出或发送缓冲错误都可能让客户端收不到。客户端以“连接关闭 + 下次登录查询事实”为最终恢复手段，不能把 logout Response 当作强持久化确认。

19. 被动断线与 60 秒重连窗口

Gate disconnect/error 通知 Watchdog, Watchdog 删除 `connections[fd]` 并 send Agent `client_closed(fd, connection_id)`。

Agent 只有在 `(fd, connection_id)` 都匹配当前绑定时才接受：

```
clear client binding
client_ready = false
pending_pushes = {}
offline_version++
capture version
schedule timeout(6000)
```

若 60 秒内重连, `bind_client` 会再次增加 `offline_version`; 旧 timer 看到 version 不一致就退出。玩家仍留在 Scene, PlayerAgent 和 realtime state 得以复用。

19.1 Grace Window 的产品语义

窗口期间需要明确：

- 角色是否继续被攻击、死亡或参与 AI？
- AOI Push 丢弃还是缓存？当前实现因无 fd 而丢弃。
- 重连后如何获得完整最新 snapshot？当前只复用 Agent，并未重新发送全量 Scene Snapshot。
- 经济操作是否继续？是否允许其他设备顶号？

“Service 没退出” 不等于 “重连恢复已完整”。生产重连必须有明确 Resync Protocol。

20. Offline Pipeline：退出并不是一个原子操作

Grace Timer 到期后，`serial(offline)` 执行：

```
check client_fd is nil
-> call Scene.leave      [yield]
-> scene_service = nil
-> call Storage.save(player) [yield]
-> call PlayerMgr.remove  [yield]
-> Agent exit
```

这条链跨三个 Owner，不存在自动事务。每一步失败留下的状态不同：

失败点	Scene	PlayerMgr	Storage	风险
leave 前	玩家仍在	映射存在	旧快照	可继续重试
leave 后、save 前	已离场	映射存在	旧快照	Runtime 与持久化分离
save 失败	已离场	映射存在	旧快照	当前代码仍继续 remove/exit
remove 失败	已离场	可能仍指向 Agent	已保存	陈旧 Service Address

20.1 当前实现的一个重要竞态窗口

`offline()` 只在开头检查一次 `client_fd`。它在 `Scene.leave` 或 `Storage.save` 上 yield 时，`CMD.bind_client` 不受 `serial` 保护，理论上新连接可能重新绑定；旧 offline coroutine 恢复后仍可能继续 remove/exit。

这说明 Generation Check 不能只放在 timer 入口。真实修复需要把 offline 设计成带 version 的多阶段状态机：每次 await 恢复后重新校验 version/client binding，或让 bind 与 offline commit 进入同一个互斥协议。第四课前不应把当前 Grace Window 宣称为生产完备。

21. Storage：同一玩家路由有序，不代表已经持久可靠

StorageMgr 用：

```
worker_index = player_id % pool_size + 1
```

把同一 player_id 稳定路由到同一 Worker。这有利于单玩家写入排序，也让不同 Worker/DB connection 提供并行度。

Memory 与 MySQL Worker 提供相同 RPC surface：load_player/save_player。但二者语义差异必须清楚：

- Memory 模式随进程退出丢失，只用于教学和确定性测试。
- MySQL UPDATE 成功才表示 DB 接受本次写入。
- 当前只有最终离线保存，没有 periodic snapshot、dirty set、WAL 或关键经济事务。
- 当前 UPDATE 没有 version 条件，旧 snapshot 晚到时可能覆盖新状态。
- StorageMgr 仍在每次访问的数据面多跳；真实项目可在 Agent 缓存 Worker Address，但要处理 Worker 重启和代号变化。

21.1 Versioned Snapshot 的最小模型

```
PlayerAgent state_version++ on durable-state mutation
```

```
save(player_id, version, snapshot)
UPDATE player
SET ..., version = :version
WHERE player_id = :id AND version < :version
```

版本防止旧写覆盖新写，但自动解决奖励、交易、充值等跨实体原子性。这类关键操作需要幂等 Operation ID、Ledger/Transaction 和明确的重试语义。

22. 一致性不是一个词：为每条数据声明模型

不同 MMO 数据需要不同承诺：

数据	合理模型示例	不能接受的结果
Scene 坐标/AOI	单 Scene 内顺序一致、服务器权威	越界或双主坐标
战斗 HP	Scene Tick/Event 顺序	重复扣血、死后继续受击
Chat/非关键 Push	At-most-once/best effort	不应阻塞核心战斗
角色普通快照	Versioned eventual persistence	旧快照覆盖新快照
金币/道具	幂等且可审计的事务/账本	重复发放或无记录丢失
登录占用	单 player_id 唯一 owner + lease/generation	两个 Agent 同时写同角色

不要对整个系统笼统宣称“强一致”或“最终一致”。应该在消息/状态边界上明确：顺序、重复、丢失、可见性和恢复方式。

23. 超时、重试和幂等：三者必须一起设计

`skynet.call` 默认没有业务 Deadline。目标只是慢时，调用 coroutine 会持续等待。真实 RPC wrapper 通常记录 deadline，但“超时返回”不代表下游操作被取消。

典型不确定结果：

```
Agent -> Storage: grant reward and save
Storage commits
Response lost / caller timeout
Agent retries
```

没有 Operation ID 时会重复发奖。正确设计需要：

```
operation_id = globally unique business identity
request      = deterministic command
storage      = remember operation result atomically
retry        = same operation_id returns same result
```

对于 move 这类高频实时指令，通常不做跨 Service 重试：新输入会取代旧输入。对于支付、邮件领奖等关键写，必须可幂等重放。重试策略由业务语义决定，不是统一捕获异常后再 call 一次。

24. 场景切换：当前未实现，但应该怎样推导协议

切图同时涉及 OldScene、NewScene、PlayerAgent、客户端和持久快照，不能写成简单：

OldScene.leave -> NewScene.enter

至少要定义：

1. PlayerAgent 设置 **TRANSFERRING** + **transfer_id**，暂停普通请求。
2. OldScene 导出可转移实时状态或提交 leave。
3. NewScene 用 transfer_id 幂等 enter，并返回 Scene Epoch + snapshot。
4. Agent 原子切换 **scene_service/scene_id** 快照。
5. 客户端收到 change-scene barrier，再接收新 Epoch Push。
6. 任一步失败时决定回 OldScene、重试 NewScene 还是安全下线。

若 OldScene 已 leave 而 NewScene enter 失败，系统处于 Saga 的中间状态，需要补偿；Skynet Service 消息不会自动提供分布式事务。

25. 故障矩阵：先定位事实，再决定恢复

现象	首要检查	可能已提交的事实	推荐证据
登录超时	Watchdog/PlayerMgr/Storage task	Agent 可能已创建	Trace + task + Manager map
登录成功但无 AOI	client_ready/pending_pushes	Scene 已 enter	Agent/Scene snapshot
move 无 Response	Agent call 与 Scene Mailbox	Scene 可能已移动	session Trace + Scene state
看到重复实体	Scene Epoch/enter/leave 顺序	Server 可能正常、View 陈旧	Packet Record/Replay
顶号踢错连接	fd + connection_id	新绑定可能被旧消息污染	Connection generation log
掉线后角色残留	Grace timer/offline task	Scene 仍有实体	task Agent + Scene stats
坐标回档	snapshot version/save result	Realtime 曾成功但未持久化	Save version + DB row
Mailbox 增长	Hot Actor/下游等待	请求仍在排队	stat + latency histogram

日志只能作为证据之一。没有 Trace ID、player_id、Service Address、connection_id、scene_id/version 的结构化上下文，很难把跨 Actor 事件还原成同一条会话。

26. 实验一：逐断点走完整登录链

LuaPanda 一次选择一个 Service，分三轮观察。

26.1 Watchdog 轮

在这些位置断点：

```
SOCKET.open  
c.state = "AUTHING"  
each alive(fd, c)  
gate.forward  
frame.write(login response)
```

记录 `c.id/state/agent`，并在 Auth call 停住期间主动关闭客户端，验证恢复后 alive 拦截旧 coroutine。

26.2 PlayerAgent 轮

断点：`load/bind_client/enter_world/client_ready/client_closed/offline`。观察玩家身份与连接身份分别由哪些字段表示；验证 snapshot 在 login Response 之前进入 `pending_pushes`。

26.3 Scene 轮

断点：`enter/move/attack/leave/spawn_monster`。每次停住先标记是否处于 yield-free commit 段，不要在共享联调环境长时间暂停 Hot Scene。

27. 实验二：用两个客户端验证顶号与 Generation

使用同一 player_id 启动客户端 A，再启动客户端 B。

预期：

```
B -> PlayerMgr returns existing Agent
B -> Agent.bind_client
    -> send old Watchdog kick(A fd, A connection_id)
    -> bind B identity
A close event arrives later
    -> Agent.client_closed(A fd, A connection_id)
    -> rejected as stale; B remains bound
```

观察 Watchdog 日志中的 fd/conn，Debug Console `list/task` 中只应有一个对应 PlayerAgent。若机器很快复用 fd，更能说明只比较 fd 不够。

把结果写成时序记录，而不是只报告“B 登录成功”。验收需要证明旧 close 没有清掉新绑定。

28. 实验三：重连窗口与当前缺口

步骤：

1. 登录并记录 PlayerAgent Address、Scene stats。
2. 直接断开客户端，不发送 logout。
3. 60 秒内重连同一 player_id，确认 Agent Address 复用。
4. 查看 Scene 是否仍只有一个玩家实体。
5. 断线期间制造其他实体变化，重连后检查客户端是否获得完整 Resync。

前四步验证当前 Grace/Generation 机制；第五步会暴露当前 Demo 没有全量重同步协议。这个实验的目标是准确描述保证边界，不是把所有现象解释成“网络偶发”。

进一步可在 Offline Pipeline 的 `Scene.leave` 后制造延迟，并同时发起重连，用确定性测试复现第 20.1 节的 race；正式修复应带 regression test。

29. 实验四：验证客户端可见顺序

在 `client/test_client.lua` 的 `wait_response` 中记录每个 Sproto packet:

```
monotonic_time
protocol_type
session or push name
entity_id
```

验证:

- login Response 在初始 `entity_enter` 前。
- attack 的 `entity_hp/monster_dead/entity_leave` 组合可被客户端状态机接受。
- 越界 move 返回失败，且后续状态快照没有采用非法坐标。
- logout Response 不能被当作强保证，连接可能先关闭。

生产 Packet Recorder 应支持脱敏、采样和离线 Replay；不要在生产永久记录 Token 或完整隐私 payload。

30. 测试金字塔：每层证明不同性质

层级	当前例子	适合证明	不能证明
Pure Lua Unit	movement/combat/AOI	算法边界与不变量	Actor 时序
Service Contract	应补充 Login/Offline tests	RPC 返回、Generation、失败路径	真实 Socket framing
Integration Smoke	<code>tests/integration/smoke.sh</code>	Gate/Proto/Scene 完整链	高并发竞态和容量
Deterministic Race	应控制 yield/barrier	重连与 offline 交错	真实延迟分布
Load/Soak	第四课建设	Mailbox、Tail Latency、泄漏	单个业务规则正确性
Record/Replay	第四课建设	线上序列复现	外部依赖真实副作用

任何生产竞态修复都应有能稳定打开原窗口的回归测试。依赖 `sleep 0.1` 碰运气的测试容易既慢又不可靠；更好的方式是注入 Barrier、Fake Clock 或可控下游 Service。

31. 业务链代码评审模板

31.1 消息和上下文

- 入口是 Socket Event、`PTYPE_CLIENT`、Lua RPC、timer 还是 fork？
- source/session/connection_id/operation_id 各自是什么？
- 哪个 Client Response 或 Push 表示客户端可见？

31.2 状态和提交

- 每个字段的 Authority、Snapshot、Cache、View 分别是谁？
- commit point 在哪一行之前/之后？
- commit 后 Response 丢失，重试会发生什么？

31.3 Coroutine

- 全部直接和间接 yield 点是什么？
- yield 前发布了什么 in-flight 状态？
- 每次恢复后重新验证哪个 Generation/Version？
- 有没有入口绕过 `skynet.queue` 修改同一状态？

31.4 失败和恢复

- 目标退出、超时、业务拒绝分别如何表达？
- 部分成功时补偿、重试、查询还是安全下线？
- 操作是否幂等？Operation ID 保存在哪里？

31.5 可观测和验证

- Trace 如何跨 Gate/Agent/Scene/Storage 传播？
- 哪些状态快照能回答“事实是否已提交”？
- 哪个 regression test 固定竞态窗口？
- 哪些 Mailbox、RPC 和 DB 延迟指标定义 SLO？

32. 当前工程已经保证与尚未保证的边界

已经清晰表达

- Gate 登录前控制面与登录后 Agent 数据面分离。
- 同 player_id 创建 Agent 受分片 coroutine lock 保护。
- fd + connection_id 拒绝典型陈旧 close/kick。
- 玩家 Client Request 经 `skynet.queue` 串行。
- Scene 拥有权威坐标、AOI 与怪物状态。
- move/attack 的 Scene commit 段无 yield。
- 登录 Response 先于初始 AOI Push 排队。
- Unit + 真实 Socket Smoke 覆盖主要 Happy Path 和越界拒绝。

仍是生产差距

- Auth 只是 dev token。
- 重连没有全量 Resync/Epoch。
- Offline Pipeline 有跨 yield 重绑窗口，需要版本化状态机与回归测试。
- 最终离线保存无法承受进程崩溃；DB 写入无 snapshot version。
- 无 RPC deadline、取消、系统化幂等和错误分类。
- 无结构化 Trace、Mailbox/延迟指标、Packet Replay。
- 无跨 Scene Transfer Saga、优雅停服与恢复协议。

课程的价值在于能明确指出这些边界，而不是用“Actor 模型”四个字跳过它们。

33. 自测题（1-5）

1. 为什么 login Response 没收到，不能证明登录没有执行？

回答必须区分 Service commit、Socket send queue 和客户端观察；说明 Response 丢失时需要查询或幂等恢复。

2. PlayerAgent 与 Scene 都保存坐标，为什么当前仍不是双主？

回答必须指出 Scene 先校验并提交 Authority，Agent 只在成功 Response 后更新持久快照。

3. AUTHING 为什么必须在 Auth call 前写入？

回答必须提到 call yield、同 Watchdog 其他消息 coroutine 和重复登录窗口。

4. Gate.forward 后 Watchdog 是否退出了连接生命周期？

回答应区分数据面和控制面：业务包直达 Agent，但 close/error/kick 仍经 Gate/Watchdog 管理。

5. serial 为什么不能自动保护 bind_client 与 offline 的交错？

回答必须说明 skynet.queue 只保护经过同一 wrapper 的调用，CMD/timer 等其他入口仍能运行。

33.1 自测题（6-10）

6. Scene.enter 返回 snapshot 时，玩家在 Scene 中是否已经存在？

是。回答还应指出这意味着 Agent/网络后续失败需要恢复或清理，不能假装 enter 未发生。

7. 为什么同 player_id 路由到同一 Storage Worker 仍不足以防止旧快照覆盖？

回答应讨论 Worker 重启、多来源写、异步重试和缺少持久 version 条件。

8. Monster respawn timer 何时需要 Generation？

当同一 Entity ID/Spawn Slot 可被重置、迁移或重复调度时，旧 timer 必须验证实例代号。

9. 为什么关键经济操作不能照搬 move 的“不重试”策略？

实时移动可被更新输入覆盖；奖励/支付必须防丢和防重，需要持久 Operation ID 与幂等结果。

10. 如何证明顶号后的旧 close 没有踢掉新连接？

回答应包含两个客户端、fd/connection_id 事件记录、同一 Agent Address，以及新连接后续请求仍成功的断言。

34. 本课源码索引

service/main.lua
dependency readiness and Gate-last startup

third_party/skynet/service/gate.lua
pre-login copy, post-login redirect, fd routing

service/gateway/watchdog.lua
connection state, alive check, login orchestration

service/player/player_mgr.lua
unique Agent mapping and sharded login lock

service/player/player_agent.lua
persistent snapshot, binding generation, serial requests, offline pipeline

service/scene/scene_mgr.lua
Scene lifecycle and create lock

service/scene/scene.lua
authoritative realtime state, AOI, combat, timer

service/storage/storage_mgr.lua
service/storage/memory_worker.lua
service/storage/mysql_worker.lua
routing, clone contract and persistence behavior

lualib/protocol/schema.lua
lualib/protocol/frame.lua
Sproto request/response/push and TCP framing

tests/integration/smoke.sh
client/test_client.lua
observable end-to-end verification

35. 本课完成标准与第四课入口

完成第三课时，你能：

1. 不看文档画出完整 login -> gameplay -> disconnect -> offline 时序。
2. 为每条状态标注 Authority/Snapshot/Cache/View。
3. 对每个 `skynet.call` 写出恢复后的 Generation/Version 检查。
4. 指出至少三个“服务端已提交但客户端可能未观察”的窗口。
5. 用两个客户端验证顶号，并解释为什么旧 close 无效。
6. 说明当前重连、离线保存和 MySQL 写入为什么仍不满足生产要求。
7. 为一个跨 Scene Transfer 写出 Saga、Epoch、幂等与补偿草案。
8. 选择 Unit、Deterministic Race、Integration、Replay 或 Load Test 验证不同性质。

第四课将把这些一致性协议置于真实运行环境：结构化日志与 Trace Context、Mailbox/RPC 延迟指标、Record/Replay、Profiling、Core Dump、GDB、压测、容量规划、灰度、优雅停服与故障恢复。第三课回答“业务事实如何保持一致”，第四课回答“当系统变慢、出错或崩溃时，怎样证明发生了什么并安全处置”。